Department of
Computer Science and Engineering

Title: Exception, Custom Exception, and use of Exception Handling keywords

Object Oriented Programming Lab

CSE 202

Green University of Bangladesh

1 Objective(s)

- Exception and Use of exception handling keywords.
- To build a User defined exception.

2 Problem analysis

The java.lang.Throwable class is the superclass of all errors and exceptions in the Java language.

2.1 Errors

Errors represent irrecoverable conditions such as Java virtual machine (JVM) running out of memory, memory leaks, stack overflow errors, library incompatibility, infinite recursion, etc. Errors are usually beyond the control of the programmer and we should not try to handle errors.

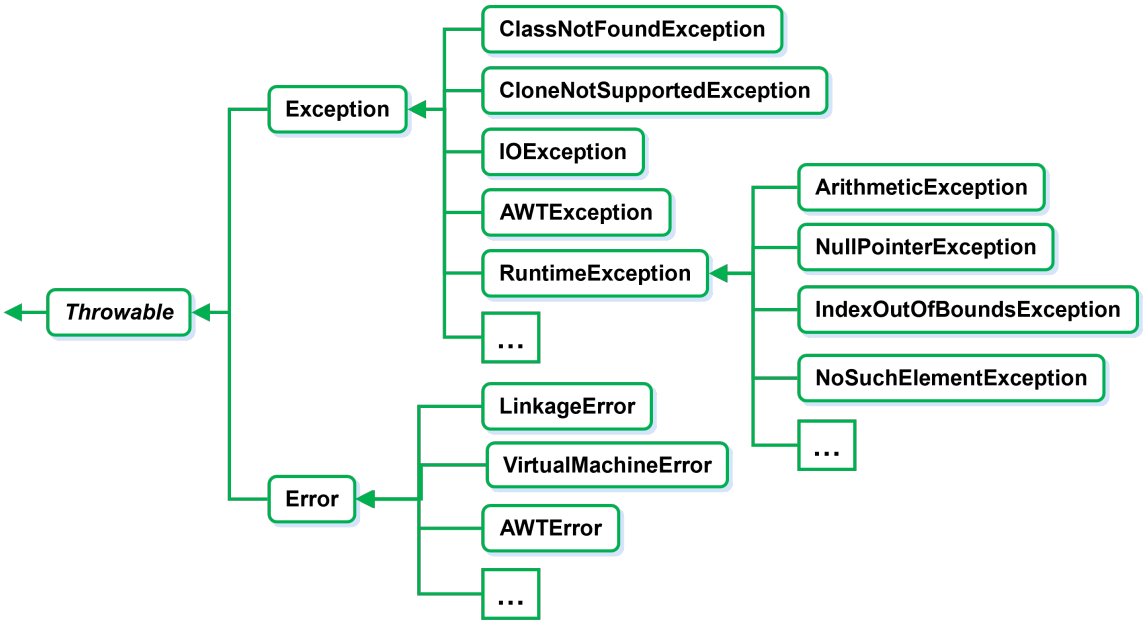

Figure 1: Java Exception class hierarchy

2.2 Exception

An exception is an unexpected event that occurs during program execution. It affects the flow of the program instructions which can cause the program to terminate abnormally. Exceptions can be caught and handled by the program. An exception can occur for many reasons, such as :

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out of disk memory)
- Code errors
- Opening a file that is unavailable

There are two common types of exception.

© Dept. of Computer Science and Engineering, GUB

2.2.1 Runtime Exception

A runtime exception happens due to a programming error. They are also known as unchecked exceptions. These exceptions are not checked at compile-time but at run-time. Some of the common runtime exceptions are:

- Improper use of an API - IllegalArgumentException
- Null pointer access (missing the initialization of a variable) - NullPointerException
- Out-of-bounds array access - ArrayIndexOutOfBoundsException
- Dividing a number by 0 - ArithmeticException

2.2.2 IO Exception

An IOException is also known as a checked exception. They are checked by the compiler at the compile-time and the programmer is prompted to handle these exceptions.

Some of the examples of checked exceptions are:

- Trying to open a file that doesn't exist results in FileNotFoundException
- Trying to read past the end of a file

When an exception occurs within a method, it creates an object. This object is called the exception object. It contains information about the exception such as the name and description of the exception and state of the program when the exception occurred. Exception handling refers to performing an action in response to an exception. For example,

- Exit program (abort)
- Deal with exception and continue
- Print error message
- Request new data
- Retry action

To understand exception handling properly, we need clear understanding of a few keywords.

2.3 Keywords

Java exception handling is managed by via five keywords:

2.3.1 try

Program statements to monitor are contained within a try block. If an exception occurs within the try block, it is thrown.

2.3.2 catch

Code within catch block catch the exception and handle it. The scope of a catch clause is restricted to those statements specified by the immediately preceding try statement.

2.3.3 finally

finally block is a block that is used to execute important code such as closing connection, stream etc. Java finally block is always executed whether exception is handled or not.

2.3.4 throw

It is possible for your program to to throw an exception explicitly using *throw ThrowableInstance*. Here, ThrowableInstance must be an object of type Throwable or a subclass Throwable.

© Dept. of Computer Science and Engineering, GUB

2.3.5 throws

If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.

2.4 Example Implementation

2.4.1 try-catch-finally block

```java
public class Main{
    public static void main(String[] args) {
        try{
            int divideByZero = 5/0;
            System.out.println("Rest of code in try block");
        } catch (ArithmeticException e) {
            System.out.println("ArithemticException => "+ e.getMessage());
        } finally {
            System.out.println("Finally block is always executed");
        }
    }
}
```

2.4.2 Exception Handling with Multiple Catch

```java
public class Main{
    public static void main(String[] args) {
        try{
            int array[] = new int[10];
            array[10] = 30/0;
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

2.4.3 throw and throws keyword

```java
public class Main{

    static void throwOne() throws IllegalAccessException {
        System.out.println("Inside throwOne").
        throw new IllegalAccessException("demo");
    }

    public static void main(String[] args) {
        try{
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught "+e);
        }
    }
}
```

© Dept. of Computer Science and Engineering, GUB

3 Implementation

- Create a java project named Lab(this also creates a package named *lab* and a file named Lab.java file)
- Rename the Lab.java to Main.java and rename the class name from "Lab" to "Main".
- Create another class under the package *lab* named **MyException** (i.e create MyException.java file). This class should define a user defined exception.
- Create another class under the package *lab* named **Test** (i.e create Test.java file). It should throw an exception of **MyException** class.
- In the **Main** class, create a try-catch block to generate and handle exception of the **MyException** class.

```java
class MyException extends Exception{
    private int a;
    MyException(int i) { a = i;}
    public String toString ()
    { return "MyException[" + a+"]";}
}

class Test{
  static void compute (int a) throws MyException{
    if(a>10)
        throw new MyException(a);
    System.out.println("Normal Exit");
  }
}

public class Main{
    public static void main(String[] args) {
        try{
            Test.compute(1);
            Test.compute(20);
        } catch(MyException e){
            System.out.println("Caught "+e);
        }
    }
}
```

4 Input/Output

Output of the program is given below.

```
Normal Exit
Caught MyException[20]
```

5 Discussion & Conclusion

This problem is focused on clarifying the basic keywords of exception handling in java and learn how to build a user-defined exception when necessary.

6 Lab Task (Please implement yourself and show the output to the instructor)

1. Create a class InvalidMarksException which extends the Exception class. The InvalidMarksException is thrown when an invalid marks (less than zero or greater than one hundred) has been entered as an input.

6.1 Base Structure

```java

class InvalidMarksException extends Exception{
    private int marks;
    InvalidMarksException(){}
    InvalidMarksException(int i){marks = i;}
    @Override
    public String toString() { }
}
```

7 Lab Exercise (Submit as a report)

The task is divided into 3 parts.

1. Create an exception class named **AgeOutOfRangeException** extended from the class Exception. This class should contain a constructor which takes the user's age (ex. 40) as parameter. Will print following message when called, "You are older than the requested age (25 years), you are 40!!!".

2. Create an exception class named **LowGpaException** extended from the class Exception. This class should contain a constructor, with no parameter. The constructor will call the Exception class constructor with the message "Your GPA is not sufficient to apply for this job (2.5)".

3. Create a main class named GPA to prompt the user to enter his/her age and his GPA. The user application for a job will be rejected either if his age is greater than 25 years or his GPA is less than 2.5. You should declare two try-throw-catch blocks; one to handle the AgeOutOfRangeException and the other to handle the LowGpaException. If the user enters acceptable age and GPA, display the message "Your application is accepted and is under study".

8 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.

9 References

https://www.programiz.com/java-programming/exceptions.

10 Appendix

If required, course instructor might provide some additional materials.

© Dept. of Computer Science and Engineering, GUB